



Chapitre 2 : Structures algorithmiques fondamentales

EC : Algorithmique

Dr Ousmane KHOUMA

Email : ousmane.khouma@uam.edu.sn

Licence 1

ES2A/ESU2A/HEEG

Année académique 2020 - 2021

Objectifs spécifiques



- Comprendre les structures conditionnelles simples ;
- Connaître les structures conditionnelles composées ;
- Appliquer l'imbrication des structures conditionnelles ;
- Maîtriser les structures itératives ;
- Maîtriser les boucles « répéter » et « tant que » ;
- Être capable d' écrire un code propre.

Introduction



L'exécution de la plupart des programmes est liée à des conditions. L'intérêt des structures conditionnelles est de tester les conditions à exécuter au cas où elles sont vérifiées ou non vérifiées. Les structures conditionnelles sont simples ou alternatives ou à choix multiples.

Les boucles, appelées les structures répétitives permettent de répéter l'exécution d'un ou de plusieurs instructions. Selon le contexte des problèmes à résoudre, nous pouvons utiliser les boucles « pour », « répéter » et « tant que ».

La structure conditionnelle simple (1/2)



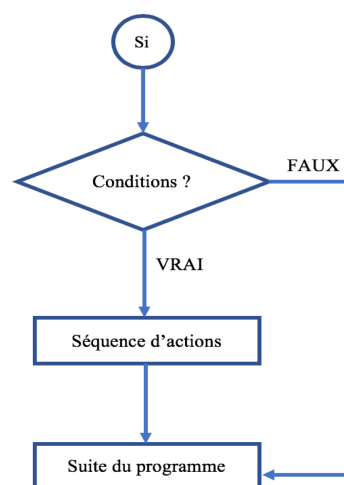
La structure conditionnelle simple permet d'évaluer une condition (expression logique à valeur 'VRAI' ou 'FAUX') et effectuer le traitement relatif à la valeur de vérité trouvée.

Syntaxe en algorithmique

Si (condition) Alors

« Séquence d'actions »

FinSi



La structure conditionnelle simple (2/2)

Exemple :

Écrire un algorithme qui demande le prénom, le nom et l'âge d'une personne puis affiche « mineur » si la personne est âgée de moins de 18 ans et « majeur » dans le cas contraire.

Solution

```
Programme categoriepersonne
var age : entier
var nom, prenom : chaine
Début
    Ecrire("Veuillez entrer le prenom de la personne :")
    Lire(prenom)
    Ecrire("Veuillez entrer le nom de la personne :")
    Lire(nom)
    Ecrire("Veuillez entrer l'âge de la personne :")
    Lire(age)
    Ecrire("Le prenom est :", prenom)
    Ecrire("Le nom est :", nom)
    Ecrire("L'âge de la personne est", age, "ans")
    Si (age<18) alors
        Ecrire("Vous êtes mineur")
    FinSi
    Si (age>=18) alors
        Ecrire("Vous êtes majeur")
    FinSi
FIN
```

La structure conditionnelle composée (1/2)

La structure conditionnelle composée permet de tester toutes les alternatives. Si le test n'a pas marché (il est faux), on va dire à l'ordinateur d'exécuter d'autres instructions.

Syntaxe

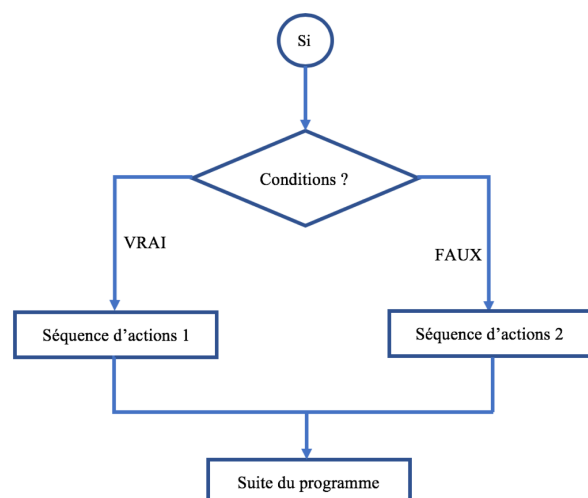
Si (condition) Alors

« séquence d'actions 1 »

Sinon

« séquence d'actions 2 »

FinSi



La structure conditionnelle composée (2/2)

Exemple :

Écrire un algorithme qui permet de saisir un entier et d'afficher « Pair » si cet entier est pair ou « Impair » si cet entier est impair.

Programme parite

```
var a : entier
Début
  Ecrire("Veuillez saisir un entier :")
  Lire(a)
  Si (a mod 2=0) alors
    Ecrire("Le nombre est pair")
  Sinon
    Ecrire ("Le nombre est impair")
FinSi
FIN
```

Exercice d'application : Écrire un algorithme qui demande deux nombres à l'utilisateur et l'informe ensuite si leur produit est négatif ou positif (on laisse de côté le cas où le produit est nul). Attention toutefois : on ne doit **pas** calculer le produit des deux nombres.

Imbrication des structures conditionnelles (1/1)

Il est possible aussi de faire un **sinon si** pour faire un autre test si le premier test n'a pas marché.

Syntaxe :

```
Si (condition 1) Alors
  « séquence d'actions 1 »
SinonSi (condition 2) Alors
  « séquence d'actions 2 »
Sinon
  « séquence d'actions 3 »
FinSi
```

Exemple

Programme etateau

```
Var Temp : entier
Début
  Ecrire ("Entrez la température de l'eau :")
  Lire (Temp)
  Si Temp =< 0 Alors
    Ecrire ("C'est de la glace")
  SinonSi Temp < 100 Alors
    Ecrire ("C'est du liquide")
  Sinon
    Ecrire ("C'est de la vapeur")
Finsi
Fin
```

Exercice d'application

Écrire un programme en algorithmique qui demande les trois paramètres a, b et c d'un polynôme du second degré, le programme donne ensuite le nombre et la valeur des racines du polynôme.

La structure conditionnelle à choix multiples (1/4)



La structure conditionnelle à choix multiples propose à l'utilisateur une liste d'options possible. Elle laisse à l'utilisateur de faire son choix. Le traitement d'une séquence dépend de la valeur saisie par l'utilisateur. La structure à choix multiples est réalisée en algorithmique par le mot clé **selon (ou suivant)**.

Syntaxe :

```
Selon (nomvariable) Faire  
    Option1 : < Action1 >  
    Option2 : < Action2 >  
    Option3 : < Action3 >  
    :  
    OptionN : < Action3 >  
Sinon  
    < Actions >  
FinSelon
```

La structure conditionnelle à choix multiples (2/4)



Exercice d'application

Écrire un programme algorithmique qui permet de simuler le fonctionnement d'une calculatrice en proposant le menu suivant :

- 1) Addition
- 2) Soustraction
- 3) Multiplication
- 4) Division entière
- 5) Reste de la division entière
- 6) Division réelle
- 7) Quitter

La structure conditionnelle à choix multiples (3/4)



Solution

Programme calculatrice

Var x, y, choix : entier

Début

```
Ecrire("Menu")
Ecrire("1: Addition")
Ecrire("2: Soustraction")
Ecrire("3: Multiplication")
Ecrire("4: Division entière")
Ecrire("5: Reste de la division entière")
Ecrire("6: Division réelle")
Ecrire("7: Quitter")
Ecrire("Veuillez faire un choix")
    Lire(choix)
Selon (choix) Faire
```

```
1...6
Ecrire("Veuillez entrer la première valeur")
    Lire(x)
Ecrire("Veuillez entrer la deuxième valeur")
    Lire(y)
1: Ecrire(x, " + ", y, "=", (x+y))
2: Ecrire(x, " - ", y, "=", (x-y))
3: Ecrire(x, " * ", y, "=", (x*y))
4: Si (y=0) alors
    Ecrire("Impossible de diviser par zéro")
Sinon
    Ecrire(x, " DIV ", y, "=", (x DIV y))
FinSi
```

La structure conditionnelle à choix multiples (4/4)



```
5: Si (y=0) alors
    Ecrire("Impossible de diviser par zéro")
Sinon
    Ecrire(x, " MOD ", y, "=", (x MOD y))
FinSi
6: Si (y=0) alors
    Ecrire("Impossible de diviser par zéro")
Sinon
    Ecrire(x, " / ", y, "=", (x / y))
FinSi
7: Ecrire("A bientôt")
Sinon
    Ecrire("Erreur, le choix n'existe pas")
FinSelon
FIN
```

La structure itérative « Pour » (1/2)

Dans un algorithme où le nombre d'itérations est connu, la boucle « Pour » doit être utilisée. Dans ce cas, il est important de connaître le nombre de fois que les actions doivent être exécutées.

Syntaxe

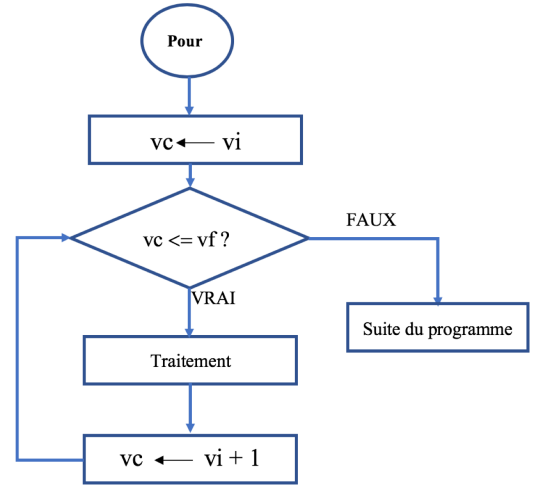
```
Pour vc allant de vi à vf Faire
    « Traitement »
FinPour
```

vc : valeur compteur de type entier
vi : valeur initiale
vf : valeur finale

Remarque:

- Une boucle peut être exécutée une ou plusieurs fois ;
- Si le pas est différent de 1, il faut ajouter l'option (pas = constante).

Pour vc allant de vi à vf par pas « valeur pas » **Faire**



La structure itérative « Pour » (2/2)

Exemple

Écrire un algorithme qui demande un nombre de départ, et qui ensuite écrit la table de multiplication de ce nombre, présentée comme suit (cas où l'utilisateur entre le nombre 7) :

Table de 7 :

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

Solution

Programme tableMultiplication

Var n, i : entier

Début

Ecrire("Entrez un nombre :")

Lire (n)

Ecrire("La table de multiplication de ce nombre est : ")

Pour i allant de 1 à 10 **Faire**

Ecrire(n, " x ", i, " = ", n*i)

FinPour

Fin

Exercice d'application

Écrire un algorithme qui demande à l'utilisateur de saisir une valeur entière positive.

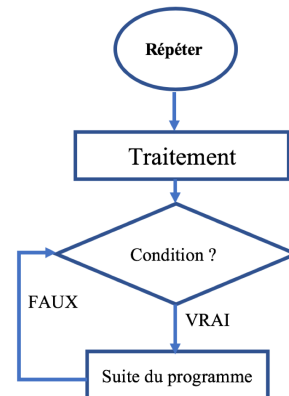
L'algorithme affiche tous les nombres pairs compris entre 1 et la valeur saisie.

La boucle « Répéter » (1/2)

La boucle « REPETER » permet d'exécuter une ou plusieurs actions.

Syntaxe

```
Répéter
    « Traitement »
jusqu'à (condition)
```



- **Condition** : condition d'arrêt et de sortie de la boucle.
- **Traitement** : action ou ensemble d'actions à exécuter tant que la condition n'est pas vérifiée.
- Si la condition est vérifiée, l'exécution du traitement s'arrête.
- **Le nombre de répétition n'est pas connu à l'avance.**
- Le **traitement** est exécuté au moins une fois quelque soit le résultat de la condition.
- La condition doit être initialisée avant le début de la boucle et doit être modifiée à l'intérieur de la boucle

La boucle « Répéter » (2/2)

Exercice d'application

Écrire un algorithme qui permet de contrôler la saisie d'une valeur entière positive. L'algorithme affiche la valeur saisie si elle est correcte sinon il affiche un message d'erreur.

Solution

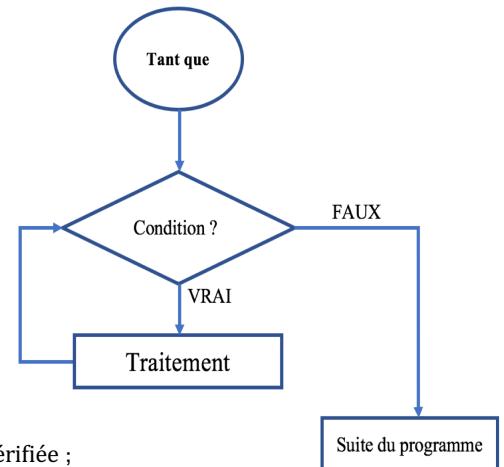
```
Programme exoapplication
Var N : entier
Début
Répéter
    Ecrire("Entrez une valeur entière positive")
    Lire (N)
    Si (N<=0) alors
        Ecrire("La valeur ",N,"n'est pas correcte")
    Sinon
        Ecrire("La valeur correcte est : ", N)
    FinSi
Jusqu'à (N>0)
Fin
```

La boucle « Tant que » (1/2)

La boucle « Tant que » est utilisée pour faire le contrôle de saisie. Elle permet de répéter l'exécution de certaines instructions (ou traitement).

Syntaxe :

```
Tant que (condition) faire
    « Traitement »
Fin Tant que
```



- **Condition** : condition de maintien de la boucle ;
- **Traitement** : action ou ensemble d'actions à exécuter tant que la condition est vérifiée ;
- Le traitement est **exécuté tant que la condition est vérifiée**, sinon on sort de la boucle ;
- Si la condition n'est pas vraie dès la première exécution, la boucle ne sera jamais exécutée (**0 fois**) ;
- **La condition doit être initialisée** avant la boucle **et modifiée** à l'intérieur de la boucle.

La boucle « Tant que » (2/2)

Exercice d'application

Écrire un programme algorithmique qui demande à l'utilisateur de saisir une valeur entière positive. Le programme affiche tous les nombres pairs compris entre 1 et la valeur saisie.

Solution

```
Programme nombrepair
Var n, i : entier
Début
    Ecrire("Entrez une valeur entière positive")
    Lire (n)
    Tant que (n<=0) faire
        Ecrire("Veuillez entrer une valeur positive")
        Lire (n)
    FinTantque
    i ← 2
    Tant que (i<=n) faire
        Ecrire(i,"est pair")
        i ← i+2
    FinTantque
Fin
```

NB :

Il faut utiliser la boucle « Tant que » ou « répéter » pour les cas où on vous demande de contrôler une saisie.